תקציר ומוטיבציה

מערכות התחברות מבוססות סיסמה חשופות במיוחד לתקיפות מקוונות כגון **brute force** ו **password spraying.** במטלה זו בניתי שרת התחברות קטן ב flask, המדמה שירות אמיתי ובחנתי באופן שיטתי את עמידותו לשתי התקפות אלו תחת שילובים שונים של מנגנוני הגנה.
השרת תומך בשלושה אלגוריתמי hashing לסיסמאות:

- **SHA 256 + SALT**
- **bcrypt**
- **Argon2**

עם 30 חשבונות בדיקה מחולקים לסיסמאות ״חלשות״, ״בינוניות״, ״חזקות״.
מעל שכבת ה hash יישמתי מנגנוני הגנה שכיחים:

- **הגבלת קצב - rate limit**
- **נעילת חשבון - account lock**
- **CAPTCHA מבוסס כישלונות גלובליים**
- **אימות דו שלבי מסוג TOTP**
- **pepper סודי בצד השרת**

לצורך הערכה הרצתי גם תקיפות brute force ממוקדות משתמש על בסיס קובצי מילון נפוצים (*קישור בסעיף מקורות), וגם תקיפות password spraying המשתמשות במספר סיסמאות מצומצם על פני כל המשתמשים.
כל ניסיון התחברות נרשם לקובצי יומן בפורמט JSONL, כולל זמן, תוצאה, סוג הגנה, ו latency של ההתחברות על בסיסם חישבתי זמני תקיפה, קצבי ניסיונות התחברות והסתברות הצלחה לפי קטגוריית הסיסמא. בנוסף ביצעתי מדידת CPU וזיכרון עבור כל אלגוריתם hash תחת עומסי ההתקפות.
הממצאים מראים כי מעבר מאלגוריתם hashing כללי (SHA256) לאלגוריתמים איטיים ומותאמי סיסמאות (bcrypt, Argon2) מצמצם משמעותית את קצב הנסיונות האפשרי לתוקף וכי מנגנוני rate limit ו account lock מגבילים היטב תקיפות brute force ישירות אך פחות יעילים מול password spraying. הכנסת TOTP כשכבת הגנה נוספת כמעט ומבטלת לחלוטין את כדאיות התקיפה, גם כאשר הסיסמא עצמה חלשה. על בסיס ניסוי זה נדון בשיקולי האיזון בין אבטחה, ביצועים ושימושיות, ומגבלות המודל שבנינו ביחס למערכות ייצור אמיתיות.

רקע תיאורטי - אחסון סיסמאות, מתקפות והגנות

מערכות אימות מודרניות נדרשות להגן על סיסמאות, משתמשים גם במקרה שבו בסיס הנתונים דולף. לשם כך, מקובל שלא לשמור סיסמאות גולמיות אלא ערכים נגזרים מהן בעיקר באמצעות פונקציות גיבוב קריפטוגרפיות ומנגנוני הקשחה. בפרק זה נסקור בקצרה את גישות האחסון וההגנות שנבדקו בעבודה.

מנגנוני אחסון סיסמאות

SHA 256 + Salt

פונקציית SHA256 היא פונקציית גיבוב קריפטוגרפית כללית אשר במקורה אינה מיועדת ספציפית לאחסון סיסמאות.

- **מנגנון ה Salt:** כדי להתמודד עם התקפות מבוססות על טבלאות מחושבות מראש (Rainbow Tables) ועם סוגיית השימוש החוזר בסיסמאות, נהוג להוסיף לכל משתמש Salt - מחרוזת אקראית וייחודית המשולבת בסיסמה לפני ביצוע החישוב. בצורה זו, גם אם שני משתמשים בחרו באותה סיסמה תוצאת הגיבוב תהיה שונה עבור כל אחד מהם.

- **חולשה עיקרית:** פונקציית SHA256 תוכננה להיות מהירה מאוד ולנצל מעט משאבי CPU וזיכרון. משמעות הדבר היא שתוקף המצויד בחומרה מתאימה יכול להריץ מיליוני ניסיונות בשניה. לכן, שיטה זו מספקת הגנה מוגבלת בלבד כנגד מתקפות brute force בקנה מידה רחב.

bcrypt

bcrypt היא פונקצית גיבוב ייעודית לאחסון סיסמאותת המציעה הגנה טובה יותר בהשוואה לפונקציות הכלליות.

- **פרמטר העלות(Work Factor):** הפונקציה כוללת Salt מובנה ופרמטר ״עלות״ הקובע את איטיות החישוב. ככל שהעלות מוגדרת גבוה יותר, כך כל בדיקת סיסמא דורשת יותר זמן מעבד.

- **היתרון האבטחתי:** מנגנון זה מאפשר להאט בכוונה את תהליך הבדיקה, כל שביצוע מיליוני ניסיונות בשניה הופך ללא ריאלי עבור התוקף. בעוד

שהמשתמש הלגיטימי ״משלם״ בהמתנה זניחה של מספר מילי שניות, התוקף נאלץ להשקיע משאבים עצומים כדי לפצח את הסיסמאות.

Argon2id

Argon2id הוא אלגוריתם מודרני לאחסון סיסמאות אשר זכה בתחרות פיצוח הסיסמאות (PHC) ונחשב לסטנדרט מומלץ.

- **עמידות לחומרה ייעודית (Memory Hard):** מעבר להיותו איטי, האלגוריתם מחייב שימוש בכמות זיכרון משמעותי עבור כל חישוב. תכונה זו מקשה מאוד על שימוש בחומרה ייעודית אשר מסתמכת על הרצת מיליוני חישובים במקביל אך מוגבלת בזיכרון זמין לכל תהליכון.

- **גמישות:** האלגוריתם מאפשר כיוון עדין של זמן החישוב,צריכת הזיכרון ומידת המקביליות בהתאם לרמת האבטחה הנדרשת ולמשאבי המערכת הזמינים

שיטות תקיפה על סיסמאות

Brute Force

מתקפה זו מנסה לעבור בצורה שיטתית על מרחב הסיסמאות האפשרי (למשל כל הצירופים באורך מסוים או שימוש ברשימת מילים גדולה) עבור **חשבון יחיד.**
כאשר מנגנון אחסון הסיסמאות הוא מהיר ופשוט (כמו SHA 256 ללא הקשחה) התוקף יכול לבדוק כמות עצומה של מועמדים בזמן קצר מה שמוביל לשבירת סיסמאות קצרות או צפויות במהירות. אלגוריתמים כמו Argon2id ו bcrypt נועדו לצמצם את קצב הניסיונות האפשרי ובכך להפוך מתקפה זו ללא כדאית.

Password Spraying

בניגוד ל Brute Force הממוקד בחשבון אחד, ב **Password Spraying** התוקף בוחר מספר מצומצם של סיסמאות נפוצות (כגון "password" או "qwerty") ומנסה אותן על פני **מספר רב של חשבונות משתמשים.** המטרה היא לאתר משתמשים בעלי סיסמאות קלות לפריצה תוך הימנעות מיצירת מספר רב של כשלונות בחשבון בודד, מה שמאפשר לעקוף מנגנוני נעילת חשבון (Account Lockout).
זוהי מתקפה נפוצה ויעילה בארגונים גדולים.

מנגנוני הגנה

כדי להתמודד עם המתקפות שתוארו לעיל, נהוג ליישם שכבות הגנה נוספות ברמת האפליקציה:

- **Rate Limit (הגבלת קצב):** מנגנון המגביל את מספר ניסיונות ההתחברות המותרים בפרק זמן נתון (למשל, x ניסיונות בדקה לכתובת IP או למשתמש). הגנה זו אפקטיבית במיוחד נגד מתקפות Brute Force מקוונות(Online).

- **Account Lockout (נעילת חשבון):** חסימה זמנית של החשבון לאחר מספר מוגדר של ניסיונות כושלים רצופים, מנגנון זה מקשה מאוד על תקיפת חשבון ספציפי, אך דורש תכנון זהיר(משך נעילה, סוגי התראות) כדי למנוע ניצול לרעה למטרת מניעת שירות (Dos).

- **CAPTCHA:** מנגנון שנועד להבדיל בין משתמש אנושי לבוט אוטומטי. לאחר מספר כשלונות, המערכת דורשת פתרון אתגר ויזואלי. הדבר מגביל משמעותית את השימוש בסקריפטים אוטומטיים מהירים.

- **TOTP (Time Based One Time Password):** אימות דו שלבי (2FA) המבוסס על קוד חד פעמי המשתנה בכל 30 שניות. גם אם התוקף השיג את הסיסמא הקבועה, ללא הקוד המשתנה ההתקפה תיכשל. שילוב זה הופך את מתקפות ה Brute Force לבלתי מעשיות כמעט לחלוטין.

- **PEPPER:** סוד גלובלי של המערכת הנשמר בקובץ קונפיגורציה מאובטח או ב HSM (ולא בבסיס הנתונים לצד הסיסמאות), בעת הגיבוב, המערכת מוסיפה את ה pepper לסיסמה ול Salt.
  יתרונו הגדול הוא במקרה של דליפת מסד הנתונים: לתקוף יהיו את ה Hash וה Salt אך ללא ה Pepper הוא יתקשה מאוד לבצע מתקפת Brute Force לא מקוונת (Offline).

המלצות ותקנים

גופים בינלאומיים מובילים כגון **OWASP** ו **NIST** מפרסמים באופן שוטף הנחיות לאבטחת זהויות ואחסון סיסמאות. ההמלצות העדכניות כוללות:

1. שימוש בפונקציות גיבוב ייעודיות ואיטיות כגון Argon2id או bcrypt
2. שימוש ב Salt ייחודי וארוך לכל משתמש.
3. יישום הגנות אקטיביות כגון הגבלת קצב, ניטור חריגות ואימות רב גורמי.

עבודה זו מתבססת על עקרונות אלו ובוחנת כיצד יישום של מנגנונים שונים משפיע בפועל על זמן התקיפה ועל עמידות המערכת.

מתודולוגיה ניסויית

סביבת עבודה:
הניסויים בוצעו בסביבת פיתוח מקומית על מחשב macOS (מעבד Apple Silicon) בעזרת שרת **fLASK** הכולל אחסון סיסמאות בגיבוב מאובטחת מנגנוני **TOTP**, **Rate Limit**, **Account Lock**, **CAPTCHA** ו **Pepper** הניתנים להפעלה וכיבוי באמצעות קובץ תצורה (config.py).
קבצי המשתמשים (users_db.json) נשמרים מקומית ומשמשים בסיס לניתוח הכמותי.

יצירת מערך המשתמשים
בהתאם לדרישות המטלה נוצרו 30 חשבונות בדיקה:

- 10 משתמשים עם סיסמאות חלשות
- 10 משתמשים עם סיסמאות בחוזק בינוני
- 10 משתמשים עם סיסמאות חזקות

הסיסמאות החלשות הן ברובן קצרות (4 תווים) וכולן ספרות כך שניתן לעבור על 10 בחזקת 4 האופציות (10000) אופציות שזו משימה די פשוטה בנוסף גם קיימת סיסמא שתוכנה "password" ל weak_user_9 שאורכה אמנם ארוך יותר, אבל כמובן שזו סיסמא נפוצה מאוד ובניסוי נראה איך במקרה הקובץ שהשתמשתי בו מכיל אותה במקום הראשון ולכן נמצאה בקלות כמעט תמיד(עוד נדון בזה אבל totp captcha עוזרים למנוע זאת).
בנוסף הוספתי את הסיסמא [redacted] (מספר ת״ז שלי) ל weak_user_7 כנדרש במטלה.
הסיסמאות בחוזק בינוני מורכבות מאורך של 8 תווים ושילוב של אותיות ומספרים שזה כבר מסבך יותר את התהליך אבל כן ניתן לראות משתמש כמו medium_user_1 שהסיסמא שלו היא "amotz123" כלומר, שילוב לא כזה מסובך שכן ניתן להגיע אליו לוגית די בקלות.
הסיסמאות החזקות כבר מכילות שילובים של סימנים מספרים ואותיות עם אורך גדול מ8 ואותן בשביל לפצח בברוט פורס בתקיפה רגילה ובלי משאבים מתאימים יקח שנים ואף מעבר להצלחה.

לכל משתמש נשמרו :שם משתמשת רמת החוזק, סיסמה מקורית(בקובץ users.json בלבד! ולא ב users_db.json) וגיבוב הסיסמא בבסיס הנתונים.
חלק מהמשתמשים הוגדרו כמשתמשי TOTP לדוגמא:
weak_user_9, medium_user_7, strong_user_2. עבור משתמשים אלו נוצר סוד TOTP ייחודי והופעל הדגל totp_enabled=true.

סוגי התקיפות

בוצעו שני סוגי תקיפה עיקריים:

- **Brute Force** - ניסיון לעבור על מרחב סיסמאות גדול עבור משתמש עד למציאת סיסמא תואמת או עד הגעה למקסימום ניסיונות (50000) או זמן ריצה שעתיים(הייתה חריגה קטנה באחת הריצות).
  במתקפה זו השתמשתי בקובץ 10k-most-common.txt שמצורף מכאן:
  https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/2025-199_most_used_passwords.txt

- **Password Spraying** - שימוש ברשימת סיסמאות נפוצות (כ 60) על פני כל 30 המשתמשים במטרה לאתר חשבון עם סיסמה חלשה אחת מתוך רבים.

לכל סוג תקיפה נכתבו סקריפטים נפרדים בפייתון attack_bruteforce.py
attack_password_spraying.py

מנגנוני הגנה

בכל ניסוי הופעל שילוב אחר של:

- פונקציית גיבוב: SHA 256+Salt, bcrypt, Argon2id
- Rate Limit - הגבלת מספר ניסיונות פר דקה למשתמש.
- Account Lock - נעילת חשבון לאחר רצף כישלונות.
- CAPTCHA -דרישת טוקן לאחר מספר כשלונות גלובלי.
- TOTP- קוד חד פעמי נוסף לאחר סיסמה נכונה.
- Pepper - סוד גלובלי נוסף לחישוב הגיבוב הנשמר כמשתנה סביבה בלבד.

בכל ריצה הופעל רק השילוב הרלוונטי תוך שמירה על יתר ההגדרות קבועות.

איסוף לוגים ומדדים

השרת רשם כל ניסיון התחברות לקבצי לוג בפורמט JSONL (שורה אחת לכל ניסיון)
עבור כל ניסיון נרשמו בן היתר: timestamp, שם המשתמש, תוצאת ההתחברות (ok, bad_password, rate_limit_block, captcha_required), ערכי ה hash והגנות שהיו פעילות.
קבצי הלוג גובשו לטבלת סיכום (log_summary.numbers / log_summary.xlsx)
הכוללת עבור כל ריצה

- סוג התקיפה
- פונקציית הגיבוב
- מנגנוני הגנה פעילים
- מספר הניסיונות הכולל
- מספר הצלחות/כשלונות וקצבי הצלחה (success_rate)
- זמן ריצה מצטבר וקצב ניסיונות לשנייה
- זמן ומספר ניסיונות עד להצלחה ראשונה
- גודל מרחב הסיסמאות (keyspace) והזמן המשוער לפיצוח מלא שלו (extrapolation)
- מדדים משוערים של שימוש ב CPU ובזיכרון כפי שנמדדו ב Activity Monitor

טבלה זו מהווה בסיס לניתוח התוצאות בפרק הבא בעוד קבצי הלוג המלאים מצורפים כנספח.

תוצאות

השוואה בין פונקציות הגיבוב (baseline, בלי הגנות)
מתוך השורות של המתקפות שבוצעו באמצעות brute force:

- **SHA 256 + Salt**

- קצב של בערך 380 ניסיונות לשנייה
- 50,000 סיסמאות נבדקות בערך ב130 שניות
- הגנה חלשה מול 0- brute force - קצב מהיר

- **Argon2id**

- בערך 25 ניסיונות לשניה
- אותה רשימה של 50000 סיסמאות בערך 30 דקות

- **bcrypt**

- בערך 4 ניסיונות לשניה
- 50,000 סיסמאות - 3 וחצי שעות

בטבלת הסיכום ניתן לראות שבמצב בסיסי ללא הגנות SHA 256 + Salt מאפשר סריקה רבה של סיסמאות בזמן קצר בעוד ש Argon2id מאט את התקיפה לכ 25 ניסיונות לשנייה bcrypt לכ 4 ניסיונות בלבד.

Brute Force לעומת Password Spraying

בניסויי **brute force** קלאסי, כל התקפה כוונה אל **משתמש בודד**.
50,000 סיסמאות מתוך קובץ מילון נבדקות על משתמש יחיד.
במקרה של SHA 256 + Salt ההרצה הסתיימה תוך כ 130 שניות בעוד שב Argon2id נדרשו עשרות דקות וב bcrypt כ 3 עד 3.5 שעות לסיום מעבר על כל 50,000 הסיסמאות. ריצות שכן הצליחו התקבלה לכל היותר **הצלחה בודדת אחת.**

אם ניקח לדוגמא את weak_user_9 את הסיסמא שלו ("password") בחרתי עוד לפני שבחרתי את קובץ הסיסמאות("10k-most-common.txt") ויצא שהסיסמא הראשונה שם היא password לכן למקרה כזה שבהרצה הראשונה נמצאה הסיסמא כלום כמעט לא יכול לעזור חוץ מ captcha ו totp.
לעומת זאת, ב password spraying עם כ 64 סיסמאות נפוצות בלבד על פני כל 30 המשתמשים התקבלו **מספר פריצות מוצלחות** לאחר כמה מאות ניסיונות לעומת brute force שדרש עשרות אלפי ניסיונות על משתמש יחיד. זה מדגים למה spraying יעיל במיוחד נגד ארגונים עם משתמשים רבים וסיסמאות חלשות.
סה״כ ניסיונות 1920 - 64 סיסמאות * 32 משתמשים בפועל
בתקיפה ללא הגנות (עם pepper כן אפשר) נפרצו 2 חשבונות:

- password weak_user_9
- 0000 weak_user_4

כלומר בערך 6.7% מהמשתמשים נפרצו אחרי רק 64 ניסיונות לכל משתמש.

מנק מבט הגנתית ההבדל המרכזי הוא:

- ב **brute force** כבר אחרי כמה עשרות או מאות ניסיונות על אותו משתמש מנגנונים כמו **Rate Limit** או **Account Lock** יכולים לזהות דפוס חריג **ולחסום** את החשבון.

- ב **password spraying**, התוקף מבצע **מספר קטן של ניסיונות לכל משתמש** ולכן הרבה יותר קשה להבדיל בין התקפה לבין טעות אנוש של משתמש ששכח סיסמה. התוצאה אצלנו היא שעם פחות מ 2000 ניסיונות כלליים התוקף הצליח להשתלט על כמה חשבונות חלשים בעוד ש brute force היה צורך בעשרות אלפי ניסיונות לכל חשבון.

הניסוי מדגיש ש password spraying היא שיטה יעילה במיוחד ב**סביבות עם הרבה משתמשים וסיסמאות חלשות/נפוצות** ושגם אם כל משתמש מוגן בhash חזק כמו Argon2id או bcrypt שימוש בסיסמאות פשוטות כמו password 0000 עדיין מאפשר לתוקף להשיג פריצות מהירות בעומס קטן יחסית על המערכת.

השפעת הגנות שונות

בנוסף להשוואה בין פונקציות הגיבוב נבחנו גם מנגנוני הגנה שונים שנוספו מעל מנגנון ההתחברות:
Rate Limit, Account Lock, CAPTCHA, TOTP, PEPPER
המבחנים בוצעו על אותם תרחישי תקיפה עם אותם משתמשים ואותן רשימות סיסמאות פעם ללא הגנות ופעמים אחרות עם שילובים שונים של מנגנוני ההגנה.

Rate Limit

במערכת שמימשתי הוגדר כך :

- MAX_FAILS_PER_USER = 5
- WINDOW_SECONDS = 60

כלומר אם משתמש מסוים מבצע יותר מ5 ניסיונות כושלים בתוך דקה השרת מתחיל להחזיר עבורו שגיאות 429 ("Too many attempts") והמתקפה על אותו משתמש נעצרת זמנית.

מהריצה והלוגים רואים ש:

- ללא Rate Limit מתקפת brute force על משתמש יחיד מגיעה לעשרות אלפי ניסיונות
- עם Rate Limit מופעל מספר הניסיונות האפקטיבי על כל משתמש מוגבל לכמה ניסיונות בודדים בכל חלון זמן.
  בפועלך זה מוריד מאוד את מספר הניסיונות לשניה שהאקר יכול להריץ אונליין על משתמש ספציפי ומאריך בצורה משמעותית את הזמן הדרוש כדי לעבור על אותה רשימת סיסמאות.

כלומר Rate Limit **לא מונע כשלעצמו password spraying** כי התוקף יכול לנסות מעט סיסמאות על הרבה משתמשים אבל הוא מקשה מאוד על brute force עמוק על חשבון בודד.

Account Lock

הוגדר כך:

- ACCOUNT_LOCK_MAX_FAILS = 5
- ACCOUNT_LOCK_WINDOW_SECONDS = 300(5 דקות)
- ACCOUNT_LOCK_DURATION_SECONDS = 900 (15 דקות)

כלומר אם למשתמש יש יותר מ5 כישלונות בתוך 5 דקות החשבון ננעל ל15 דקות והשרת מחזיר 423 (Account is locked)

מהבדיקות:

- ברגע שהסף נפרץ, המשתמש ״יוצא מהמשחק״ לתקופה משמעותית. גם אם התוקף י שיך לנסות השרת כבר לא בודק סיסמאות עבורו.
- יעיל במיוחד נגד brute force: במקום עשרות אלפי ניסיונות על אותו משתמש, ההתקפה נעצרת אחרי קומץ ניסיונות, ואז צריך להמתין 15 דקות כדי להמשיך.

מצד שני ברור מהמודל שזה מנגנון שיכול להיות רגיש ל Dos אם תוקף עושה spraying אגרסיבי הוא עלול לנעול למשתמשים לגיטימיים את החשבון וזה בעצם ה trade off - אנחנו מקבלים אבטחה מוגברת אך ״משלמים״ במניעת שירות ללקוחות.

שילוב Rate Limit + Account Lock

כאשר שני המנגנונים פועלים יחד על אותו משתמש:

- **Rate Limit** מגביל את הקצב המיידי (כמה ניסיונות בדקה)
- **Account Lock** קובע ״תקרת זכוכית״ לכמות הכישלונות בתוך חלון זמן רחב יותר ואחריו החשבון נחסם ל 15 דקות

בריצות עם השילוב הזה רואים שהמתקפות כמעט ואף פעם לא מגיעות ל 50,000 ניסיונות כמו במצב ה baseline במקום זה, מספר הניסיונות קטן מאוד וחישובי ההשלכה מראים שהזמן התיאורטי ל full key space עבור brute force אונליין הופך לבלתי מעשי ימים, חודשים או שנים. תלוי בגודל מרחב הסיסמאות.

CAPTCHA

הופעל במערכת ברמה הגלובלית

- CAPTCHA_FAIL_THRESHOLD = 30

אחרי 30 כישלונות גלובליים captcha_active = True
מרגע שזה קורה כל ניסיון התחברות ללא captcha_token מחזיר שגיאת 403 captcha_required.
בלוגים של המתקפה רואים בשלב מסויים:

- הקוד מתחיל לקבל רצף אחיד של תשובות captcha_required בלי קשר לסיסמה שנשלחת.
- בשלב הזה הסקריפט האוטומטי של התוקף כבר לא מתקדם, הוא לא פותר את CAPTCHA ולכן כל הניסיונות הבאים לא באמת בודקים סיסמאות.

לסיכום CAPTCHA חותך את התקיפה האוטומטית ברגע שיש כמות חריגה של כישלונות ומכריח את התקוף לעבור לפעולה ידנית/אינטראקטיבית.

TOTP

המערכת הופעל TOTP עבור מספר משתמשים מוגדר מראש

- בשלב ה login הרגיל אם הסיסמא נכונה אבל totp_enabled=True השרת מחזיר שגיאה 401 TOTP required.
- רק קריאה ל login_totp עם סיסמה נכונה + קוד totp תקף חוזר עם ok_totp

מהניסויים:

- ב brute force על משתמש מוגן ב totp ברגע שהסיסמה נכונה ההתקפה לא מסתיימת בהצלחה. במקום "ok" מתקבל "TOTP required"
- הסקריפט שלי כמו של תוקף פשוט לא יודע לייצר totp נכון, ולכן גם סיסמה נכונה לא מספיקה כדי להשלים את הפריצה.(מצורף גם סקריפט שיודע ובודק את הזמנים של totp בקובץ test_totp.py

זה מדגים בפועל את ההבדל בין **הקשחת הסיסמה עצמה** לבין הוספת **גורם נוסף בלתי תלוי.** גם אם התוקף יצליח לעקוף את על ההגנות על הסיסמה, ללא גישה לטלפון/אפליקצית totp שלו הוא יתקע.

Pepper

הוגדר במערכת כערך שנמשך ממשתנה סביבה.

- PEPPER = os.environ.get("PEPPER, " ")

הוא **לא נשמר בבסיס הנתונים,** אלא רק בצד השרת
בתהליך הגיבוב הסיסמה עוברת קודם שרשור עם ה pepper ואז נשלחת לפונקציית הגיבוב.

מהניסויים עצמם:

- מבחינת **זמן ריצה אונליין** - לא נצפתה כמעט השפעה!. קצבי הניסיונות לשניה עם ובלי pepper דומים מאוד כי מדובר על הוספת מחרוזת קבועה לצד הסיסמה לפי הגיבוב
- כלומר pepper לא מיועד להאט את brute force על API חי אלא להגן מפני תרחיש שבו בסיס הנתונים של הסיסמאות דולף.

המשמעות:

- ללא pepper אם התוקף השיג את בסיס הנתונים (hash + salt) הוא יכול להריץ brute force אוף ליין על מחשב חזק.
- עם pepper גם אם כל בסיס הנתונים דלף החישובים אוף ליין שלו לא יעילים כי חסר לו את הסוד הגלובלי שנמצא רק בשרת - הוא חייב עכשיו לפרוץ גם את השרת עצמו כדי להתקדם.

דיון

התוצאות שהתקבלו בניסויים מדגימות היטב את המתח הקלאסי בין **אבטחה** לבין **ביצועים וחווית משתמש.**

1. פונקציות גיבוב:

- מהיר = חלש
- איטי = חזק

במצב הבסיסי ללא הגנות נוספות נצפתה תמונה ברורה

- **SHA256 + Salt:** תצורה זו מאפשרת קצב גבוה מאוד של ניסיונות לשנייה. כתוצאה מכך מתקפת Brute Force על משתמש יחיד יכולה לסרוק עשרות

אלפי סיסמאות בפרק זמן קצר יחסית. עבור המערכת זוהי בחירה נוחה מבחינת ביצועים ומשאבים, אך היא מעניקה יתרון משמעותי לתוקף במיוחד כאשר מרחב הסיסמאות מצומצם או כאשר המשתמשים בוחרים סיסמאות חלשות.

- **Argon2id**: פונקציה זו מאטה את קצב הניסיונות לשניה בסדר גודל שלם בהשוואה ל SHA256 אף על פי שביצוע Brute Force עדיין אפשרי טכנית, "מחיר הזמן" שהתוקף נדרש לשלם הופך למשמעותי, בפרט כאשר התקיפה מתבצעת באופן מקוון דרך ממשק API.

- **bcrypt:** בהגדרות שנבחרו לניסוי bcrypt הוכח כאיטי מבין השלושה. הוא מוריד את קצב הניסיונות לשניה לרמה כזו שביצוע 50,000 ניסיונות אורך שעות ארוכות. בתרחיש אמת הכולל מרחב סיסמאות גדול יותר שימוש ב bcrypt, הופך את מתקפת ה BRUTE FORCE המקוונת לבלתי מעשית.

מסקנה: בחירה נכונה של פונקציית גיבוב והדרת פרמטרים בצורה יעילה מהווה שכבת הגנה קריטית הן בדליפת בסיס נתונים והן בפני מתקפות מקוונות איטיות.

2. Brute Force לעומת Password Spraying:

השוואה בין 2 שיטות התקיפה חושפת הבדלים מהותיים:

- **Brute Force:** במתקפה ממוקדת על משתמש יחיד נדרשים עשרות אלפי ניסיונות ולעיתים יותר כדי להשיג סיכוי ממשי לפיצוח, גם אם בסופו של דבר הניסיון נכשל, שיטה זו נחשבת ל"רועשת" מאוד, היא מייצרת כמות עצומה של כשלונות על חשבון בודד ולכן קלה יחסית לזיהוי וחסימה באמצעות Rate Limit ו Account Lock.

- **Password Spraying:** בניסוי זה, שימוש ברשימה מצומצמת של כ64 סיסמאות נפוצות בלב הניב מספר הצלחות וזאת לאחר כמה מאות ניסיונות בודדים שפוזרו על פני כל 30 החשבונות.
  מכאן נובע כי Password Spraying הוא וקטור תקיפה חסכוני, שקט וריאלסטי יותר. התוקף מנצל את הנטייה האנושית לבחור סיסמאות

חלשות. עבור ארגון המונה מאות או אלפי משתמשים אפילו אחוז הצלחה בודד עלול להוביל לחדירה קטסטרופלית.

מסקנה: גם בשימוש פונקציית גיבוב חזקה ביותר, אם מדיניות הסיסמאות בארגון היא חלשה מתקפת Password Spraying תצליח לעקוף את ההגנות במהירות.

3. השפעת מנגנוני ההגנה:

מניתוח הלוגים והמדידות עולה תרומתו הייחודית של כל מנגנון:

- **Rate Limit:** מוריד דרמטית את קצב הניסיונות האפשרי על משתמש ספציפי.
  הוא הופך את ה Brute Force המקוון לכלי איטי ולא יעיל תוך הפרעה מינימלית למשתמש לגיטימי שטועה בסיסמא מעת לעת.

- **Account Lock:** יוצר ״קיר״ הגנה אפקטיבי. לאחר מספר כשלונות החשבון ננעל לפרק זמן משמעותי מה שעוצר כמעט לחלוטין מתקפת Brute For.
  החיסרון הוא הגברת הסיכון למתקפות מניעת שירות DOS מכוונות נגד משתמשים.

- **CAPTCHA:** יעיל בעיקר כנגד מתקפות אוטומטיות רחבות היקף. כאשר המערכת מזהה סף כשלונות גלובלי היא מחייבת מענה לאתגר ובכך תוקעת סקריפטים אוטומטיים עד למעבר למצב אינטראקטיבי אנושי.

- **TOTP:** מספק שכבת הגנה מסוג שונה. גם אם הסיסמה נחשפה התוקף נותר חסר אונים ללא הקוד החד פעמי או המכשיר הפיזי.

- **Pepper:** אינו משפיע על זמני התקיפה המקוונת אך הוא קריטי בתרחיש של דליפת בסיס הנתונים. ה Pepper ״שובר״ את היכולת של התוקף לבצע Brute Force לא מקוון.
-

השילוב המומלץ: ההגנה המיטבית מתקבלת משילוב של פונקציית גיבוב איטית (**Argon2id/bcrypt**) מנגנוני בלימה (**Rate Limit +Account Lock**) אימות TOTP לחשבונות רגישים ושימוש ב Pepper להגנה מפני דליפות מידע.

4. האיזון בין אבטחה לשימושיות:

ממצאי הניסוי מראים כי הפעלה מקסימלית של כלל ההגנות משפרת את העמידות אך גובה מחיר:

- הארכת זמן ההתחברות לכל משתמש (עקב הגיבוב האיטי)
- נעילות שווא של משתמשים תמימים ששכחו את סיסמתם
- מורכבות תפעולית בניהול TOTP ושחזור חשבונות

לכן, לא נכון להפעיל את הכל באופן עיוור יש לבחור שילוב מאוזן:

- בחירת **Argon2id/bcrypt** עם פרמטרים בינוניים.
- הגדרת **Rate Limit** מתון (למשל 5-10 ניסיונות בדקה)
- החלת TOTP כחובה לפחות על משתמשים בהרשאות גבוהות
- שילוב מערכות ניטור והתראה לזיהוי דפוסים של Password Spraying בזמן אמת.

שיקולים אתיים

חשוב להדגיש כי עבודה זו בוצעה תחת מגבלות אתיות מחמירות ובסביבה מבוקרת:

- ההתקפות בוצעו אך ורק על **שרת ייעודי** שפותח למטלה זו ללא מעורבות של מערכות חיצוניות או משתמשים אמיתיים.
- בסיס הנתונים (user_db.json) הכיל נתונים סינתטיים בלבד והלוגים שנאספו אנונימיים לחלוטין.
- מטרת העבודה היא לימודית הגנתית בלבד: הבנה זיהוי והקשחה של מערכות.

יש לזכור כי העולם האמיתי לוגים של מערכות אימות מכילים מידע רגיש (כתובות IP, מזהי משתמש, זמני פעילות) ולכן נדרש לנהל אותם בזהירות רבה תוך הגבלת גישה שמירה על אנונימיות ועמידה ברגולצית הגנת הפרטיות.

סיכום ומסקנות

בעבודה זו פותח שרת אימות ניסיוני המדמה סביבת אמת עליו יושמו ונבחנו שלוש פונקציות גיבוב ומגוון מנגנוני הגנה. ניתוח הלוגים מהתקיפות השונות הוכיח כי פונקציות גיבוב איטיות מאטות משמעותי את התוקף, אך אינן מספקות מענה מלא לבעיית הסיסמאות החלשות. שילוב של מנגנוני האטה ובלימה מצמצם את יעילות התקיפה האוטומטית ומספק שכבות הגנה קריטיות נוספות.
המסקנה העיקרית היא שאבטחת מערכות אימות מחייבת גישה מאוזנת של אלגוריתמים חזקים, מדיניות סיסמאות נכונה, מנגנוני הגנה אקטיביים ושמירה על חווית משתמש סבירה.

מקורות

- 10,000 סיסמאות נפוצות:

https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/2025-199_most_used_passwords.txt

- OWASP Password Storage Cheat Sheet
- NIST SP 800-63B